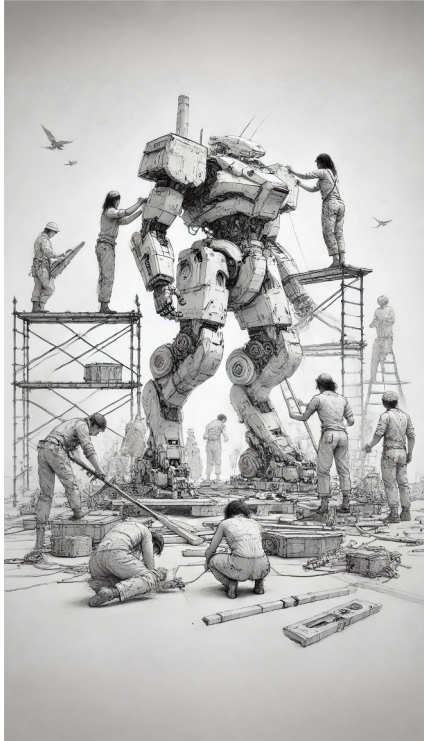




Deep Learning

Methods and applications

Convolutional Neural
Networks

Vincent BARRA



<https://vbarra.github.io>



vincent.barra@clermont-auvergne-inp.fr



- 
- 1 What _{are} **Convolution** kernels ?
 - 2 What are Convolutional networks ?
 - 3 Applications of CNN

1

What _{are} **Convolution kernels** ?

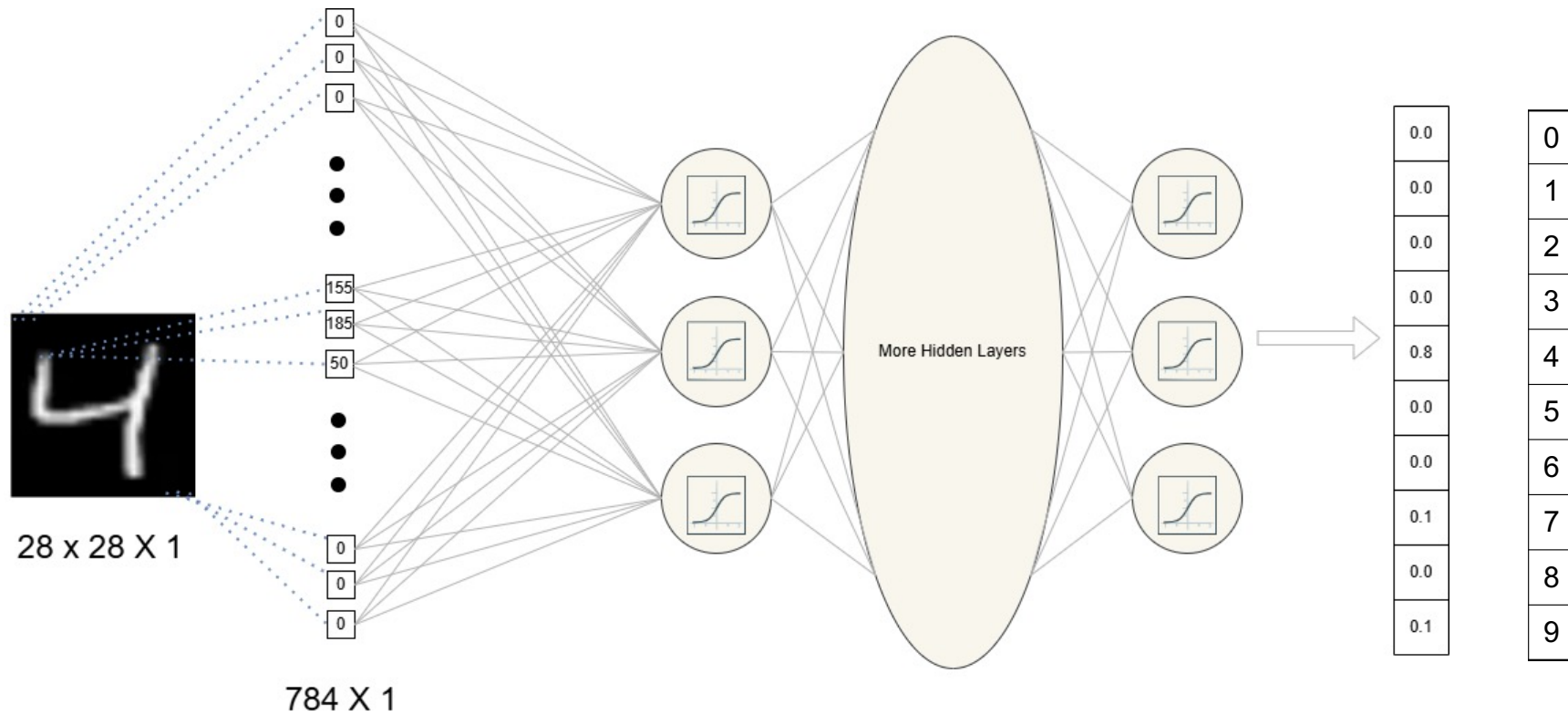
2

What are Convolutional networks ?

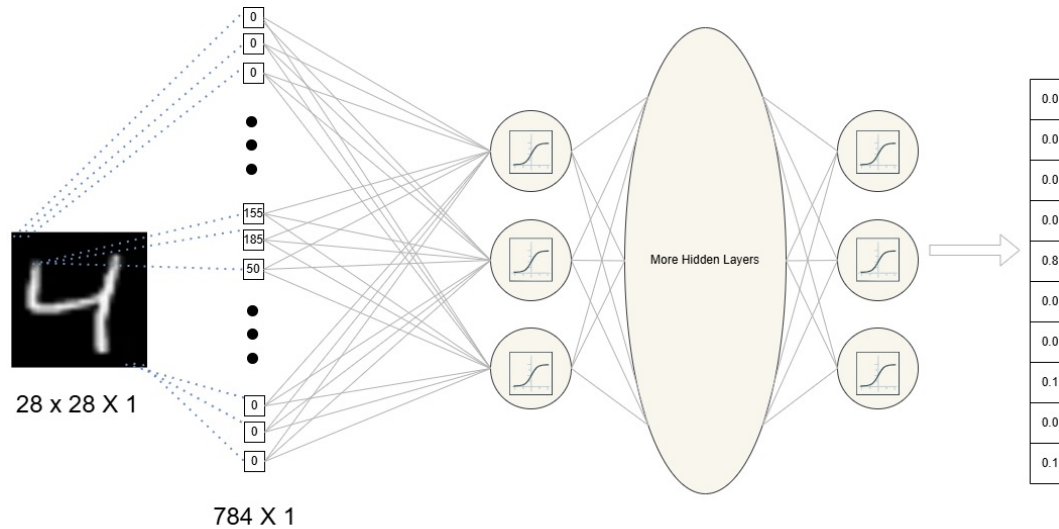
3

Applications of CNN

Using MLP on images



Using an MLP on images

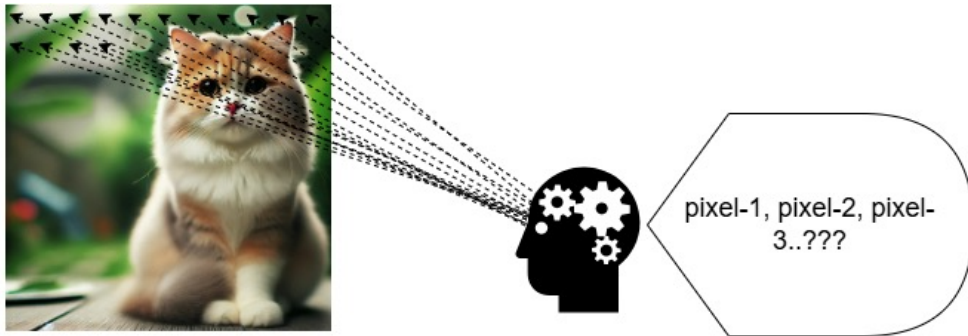


Every pixel is connected to every first layer's neuron :

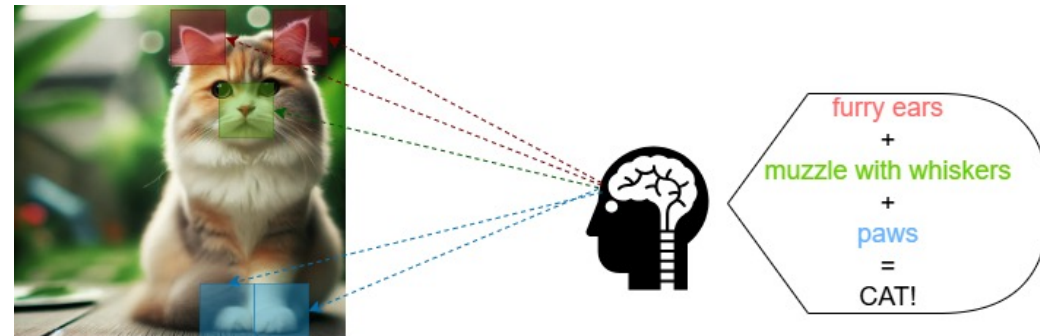
- High number of parameters!
- Some connexions are irrelevant
- Algorithm is too pixel-dependant : hard to optimize with an MLP

How do humans « classify » an image ?

Strategy 1



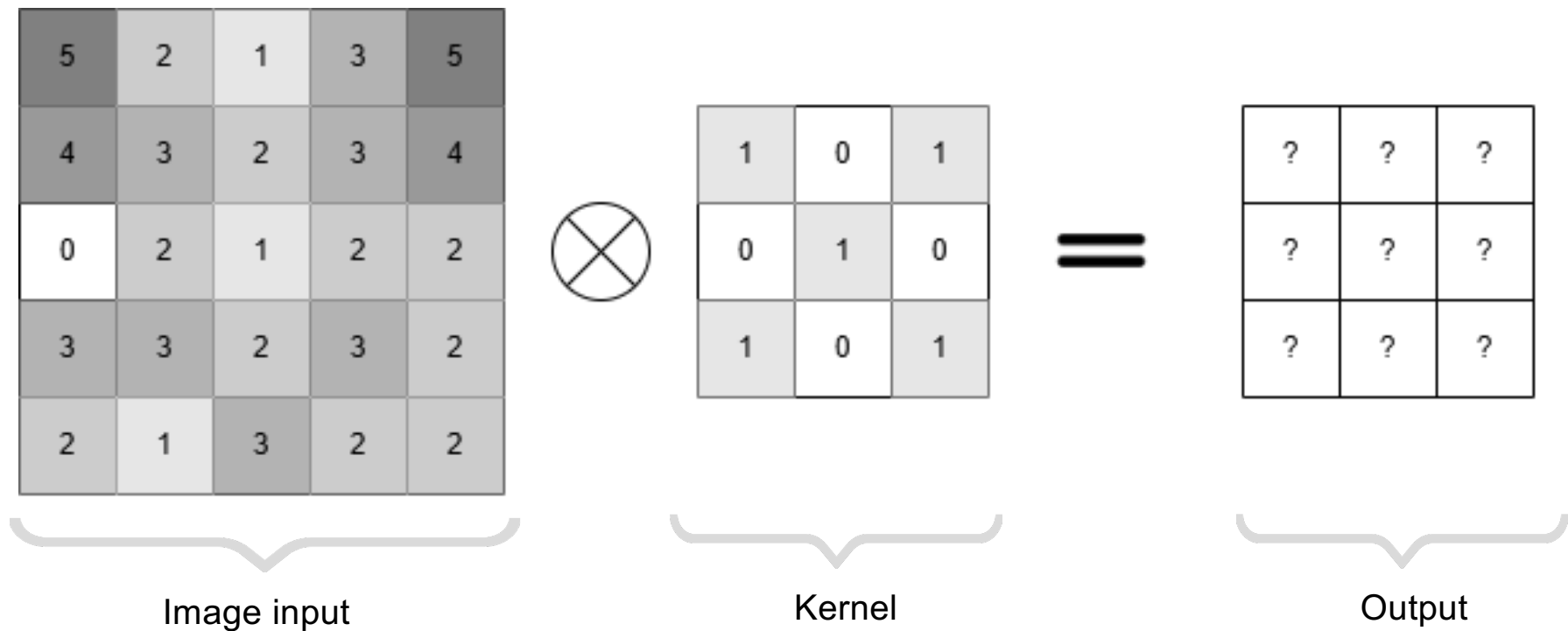
Strategy 2



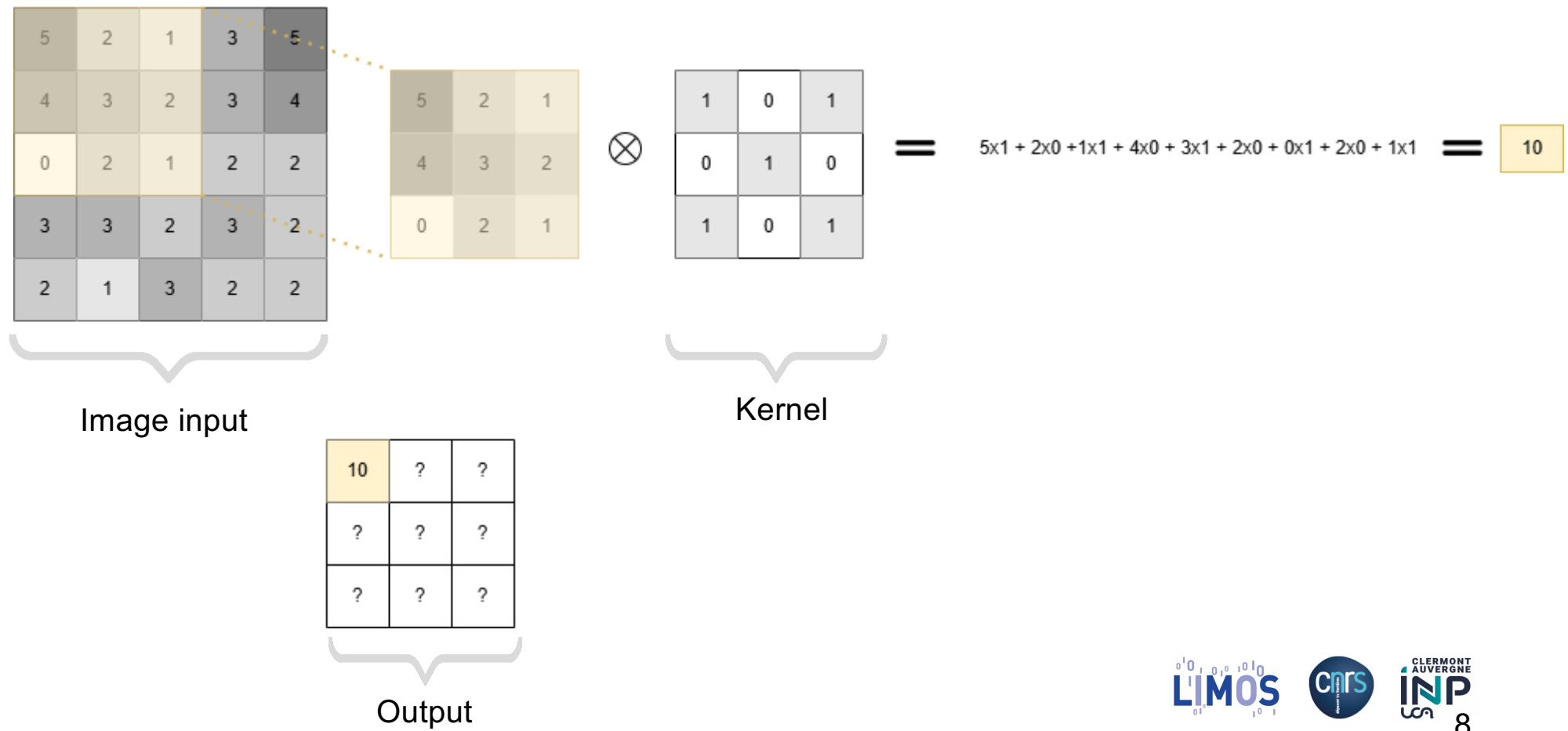
Idea : use **features detectors**

Principle of convolutions

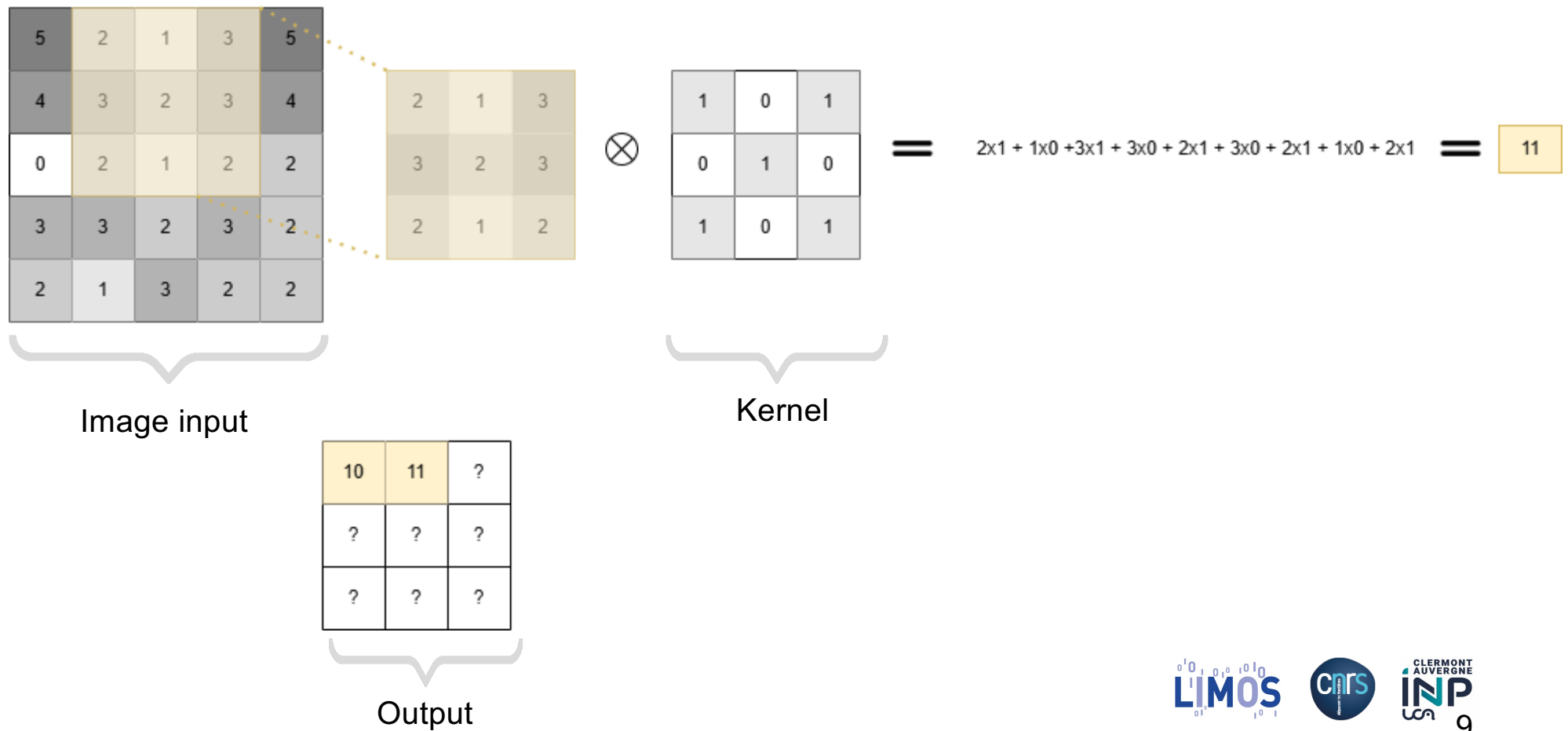
2D convolution



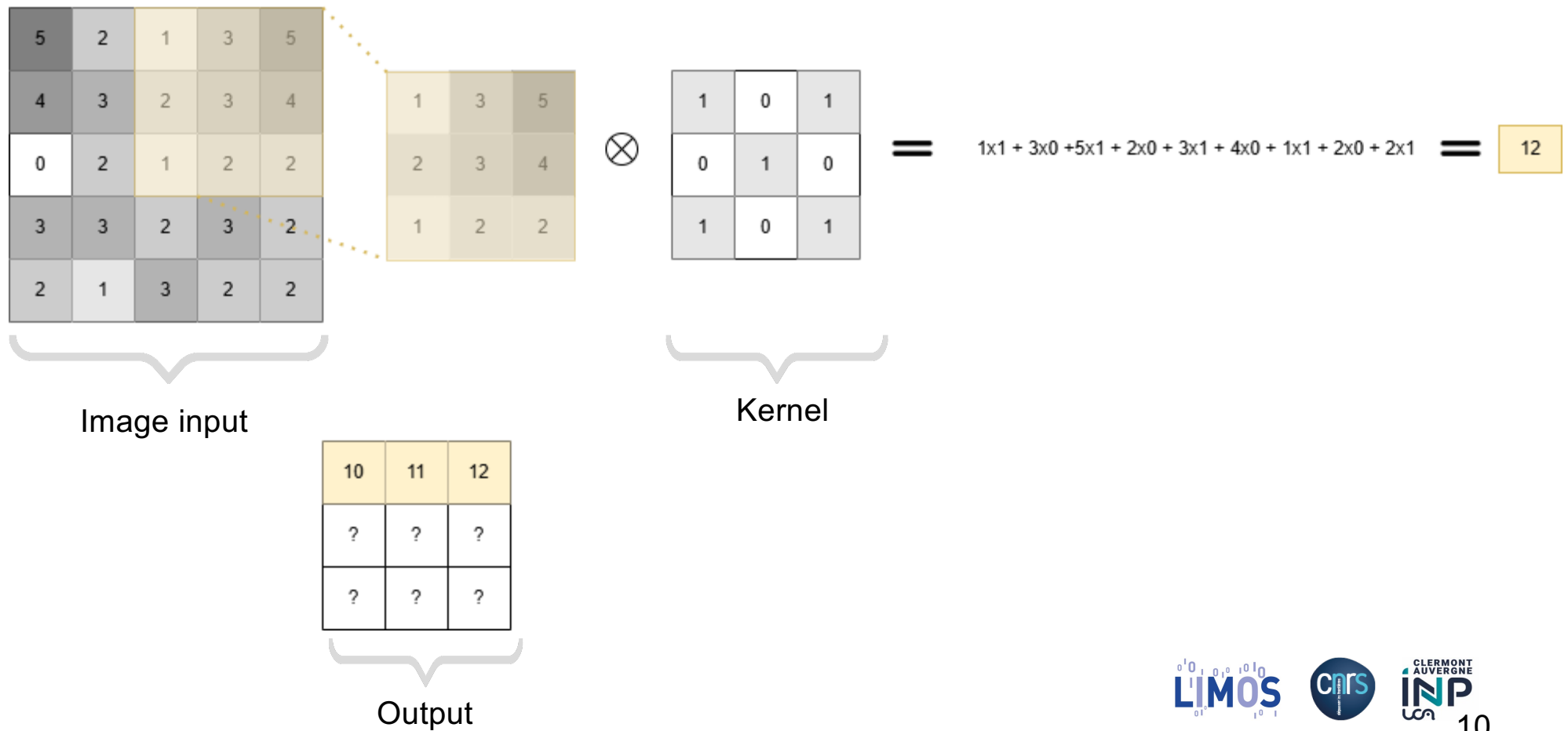
Principle of convolutions



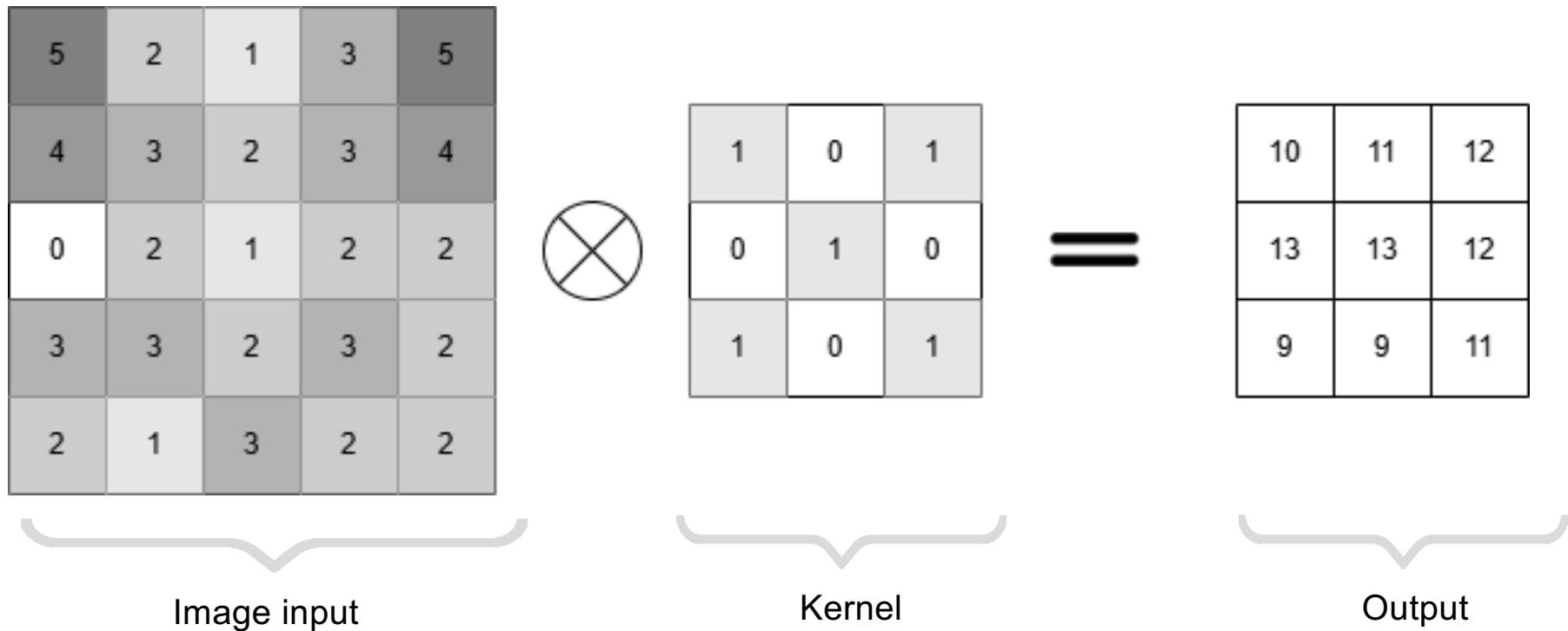
Principle of convolutions



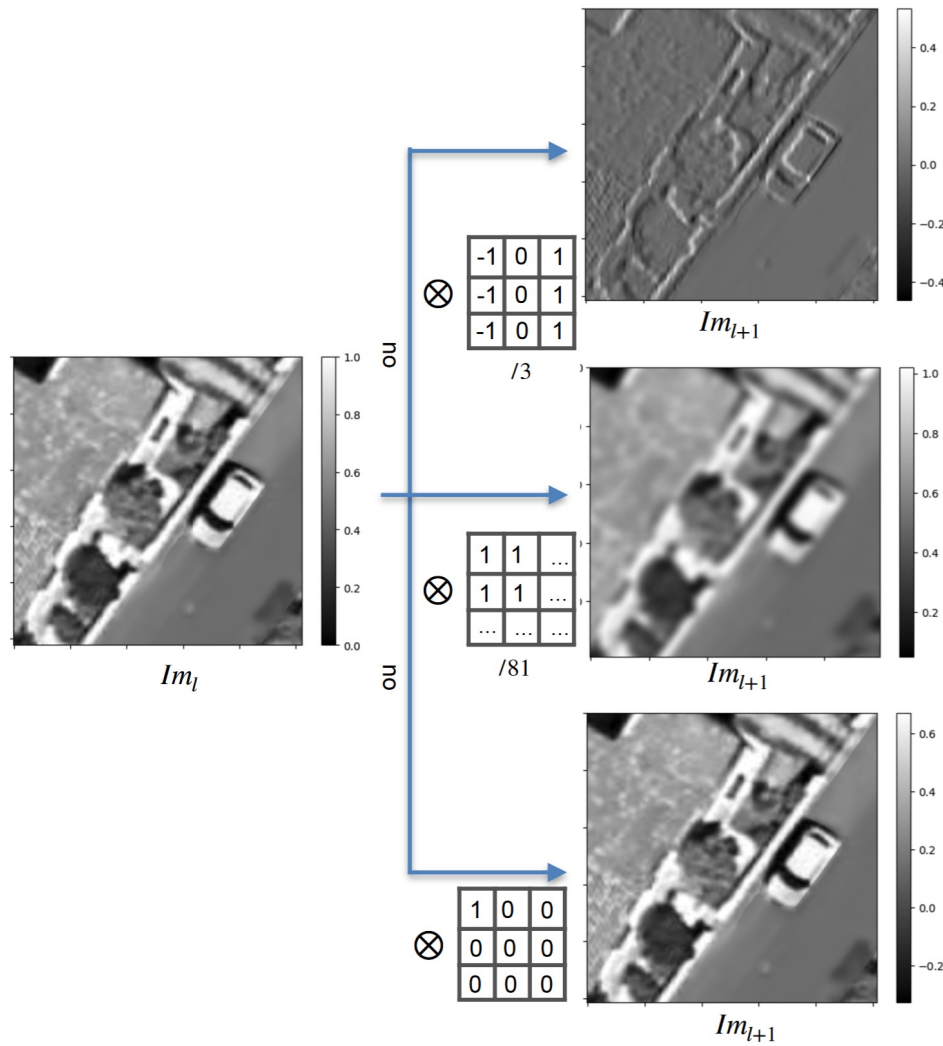
Principle of convolutions



Principle of convolutions



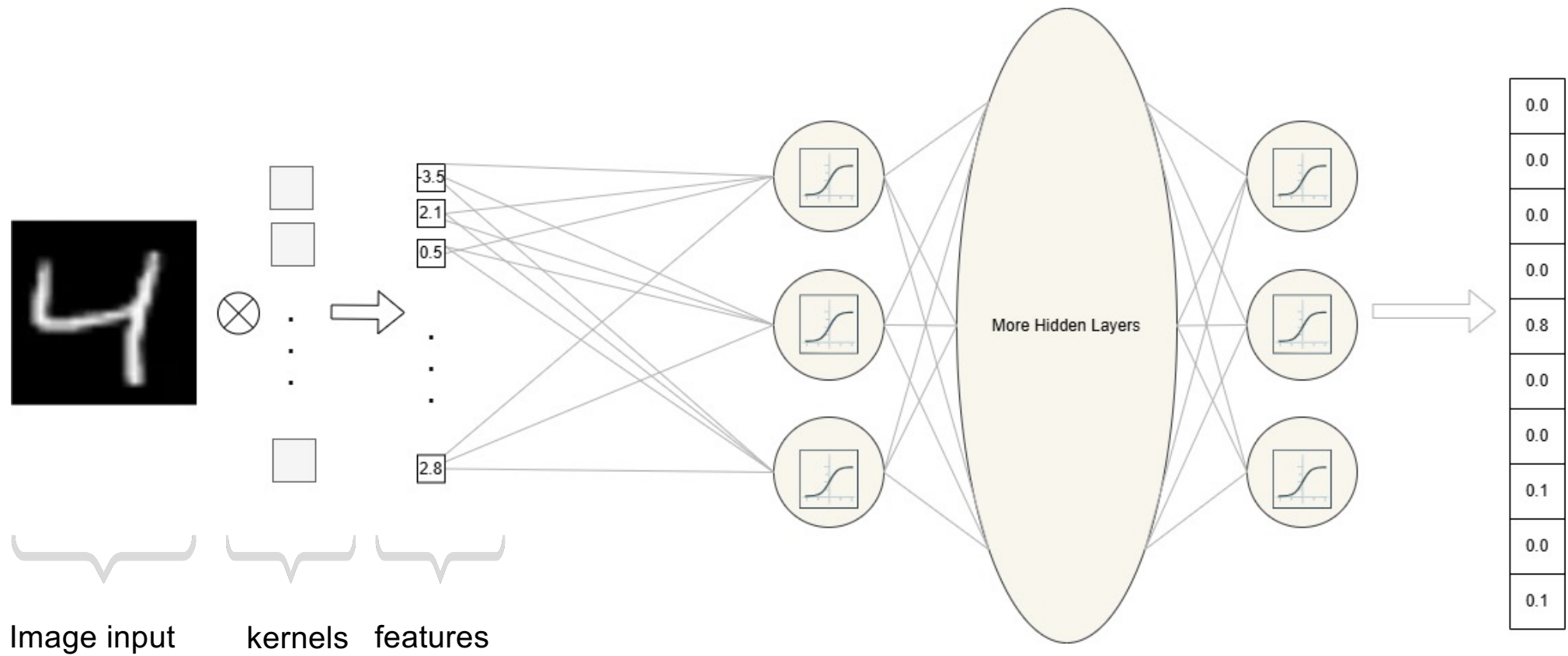
Principle of convolutions



Lot of known kernels :

- edge detectors
- shapes : curves, squares...
- textures

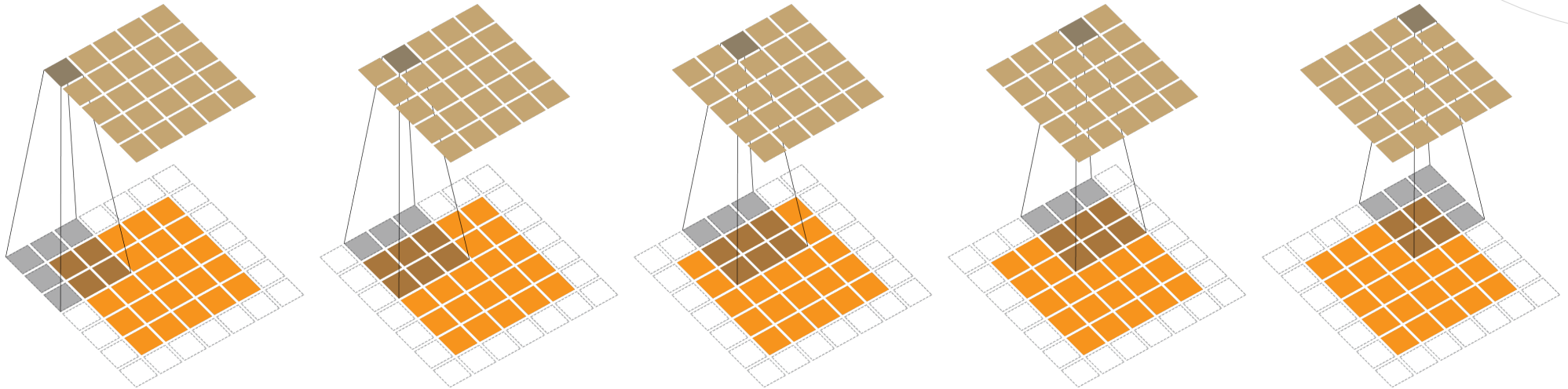
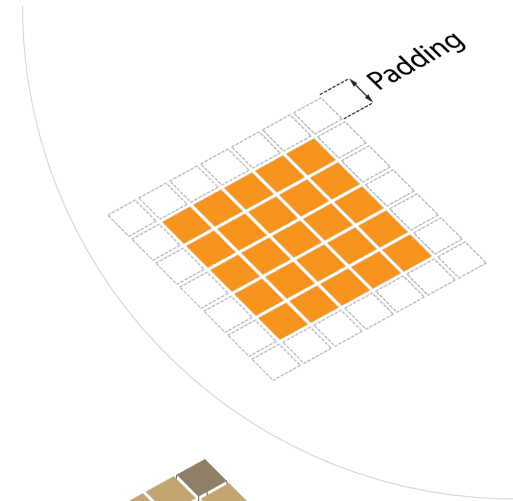
CNN : a method to improve image entries in MLP



Convolution parameters

Parameters of a convolutional layer :

Padding



Input size = N

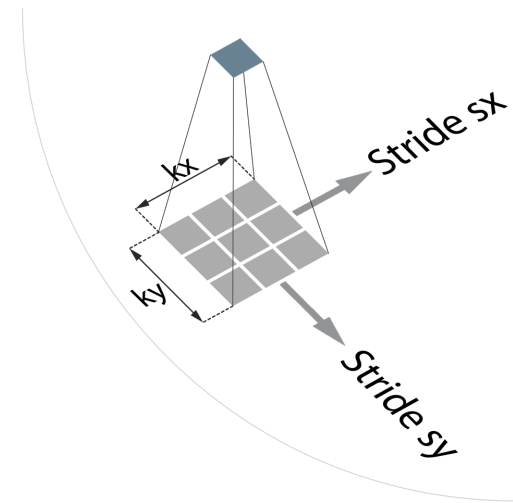
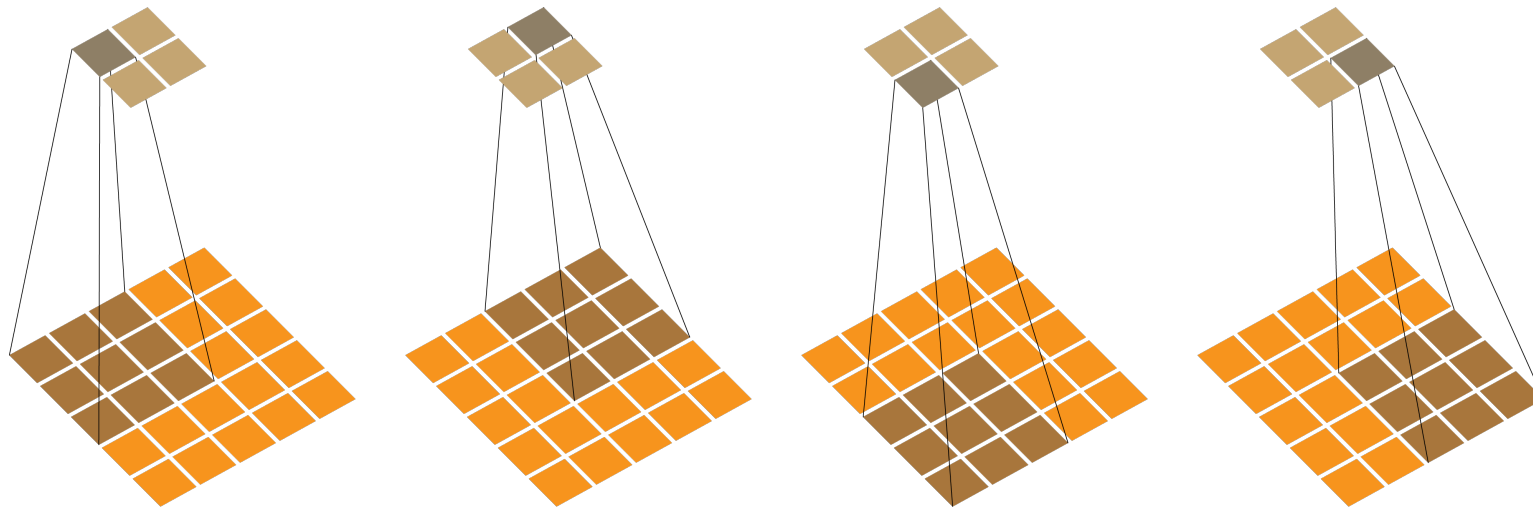
Filter size = F

Padding = P

Output size = $N - F + 2P + 1$

Convolution parameters

Parameters of a convolutional layer : **Strides**



Input size = N

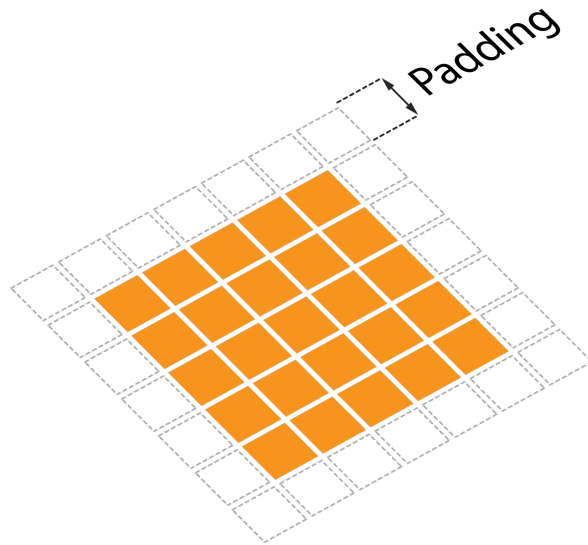
Filter size = F

Padding = P

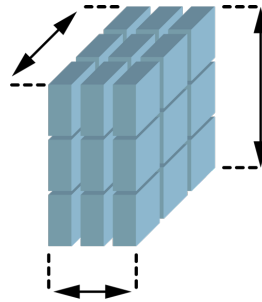
Stride = S

Output size = $\frac{N-F+2P}{S} + 1$

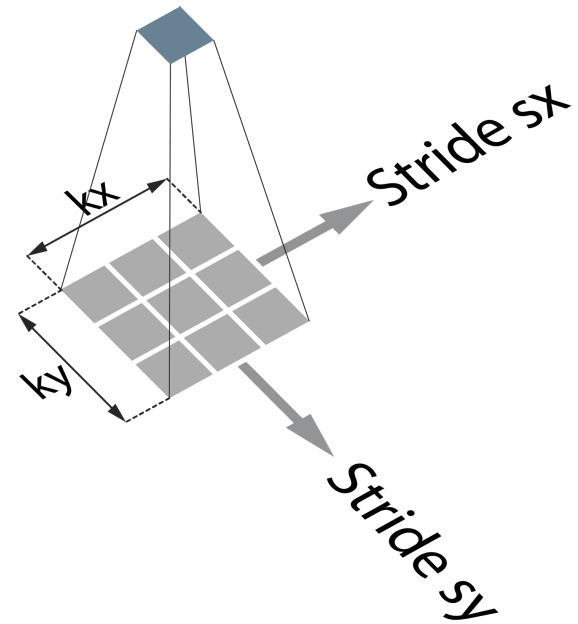
Convolution parameters



Padding



Kernel size



Strides



1 What _{are} **Convolution** kernels ?

2 What are Convolutional networks ?

3 Applications of CNN

Known convolutions are not enough

Convolution kernels are great features detectors BUT :

- It's difficult to comprehensively enumerate all the features.
- What if some features are beyond human imagination ?

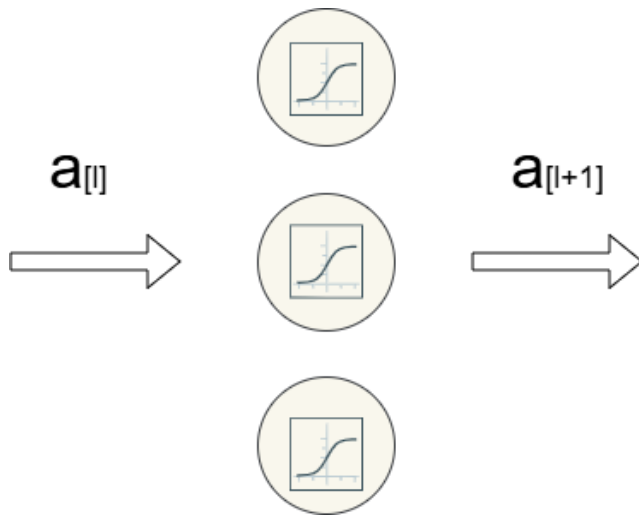
Idea : **Learn the convolution kernels** instead.

= Train a model capable of learning which kernels to be used !

Parameters of MLP vs CNN

MLP

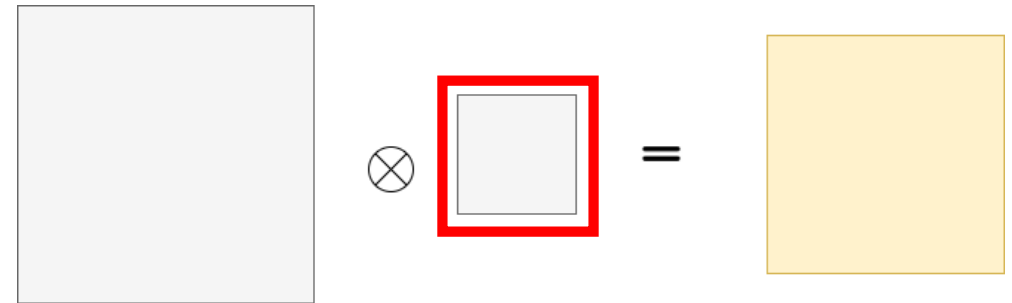
To go forward from a layer to another one :



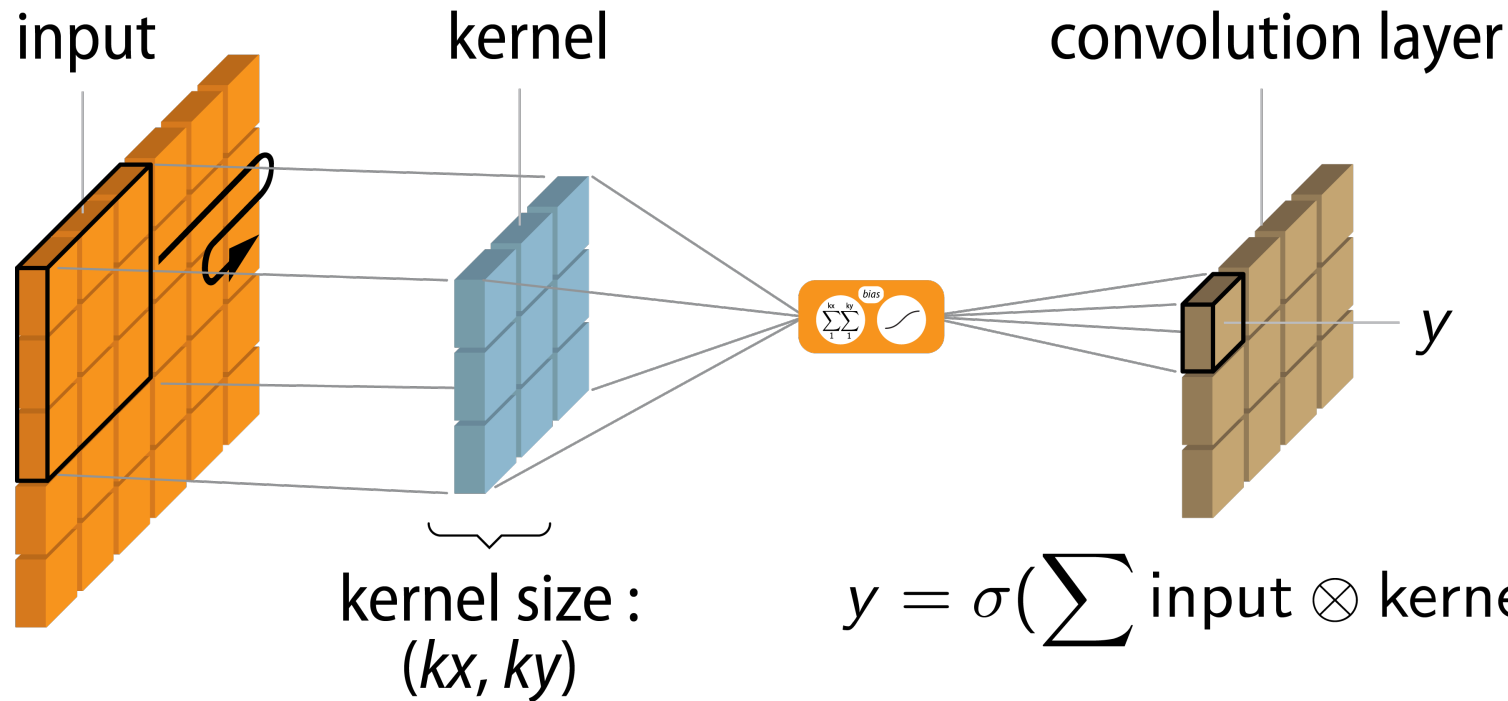
$$a^{[l+1]} = \sigma(a^{[l]} \times W^{[l]} + b^{[l]})$$

CNN :

The **values in the filters** are the parameters



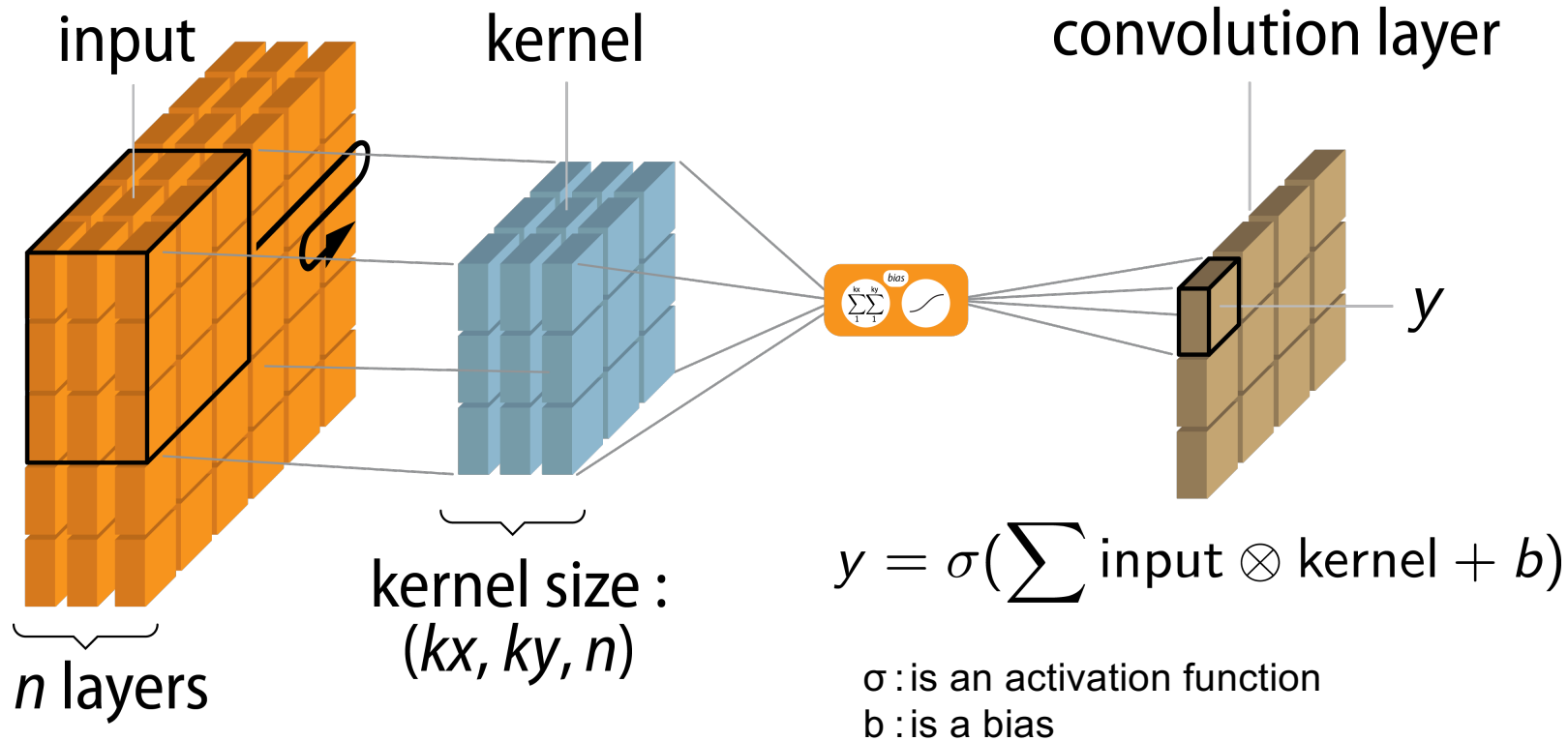
Convolutional layers : 1 channel input



σ : is an activation function
 b : is a bias

Number of parameters of a convolutional layer : $kx \cdot ky + 1$

Convolutional layers : 3 channels input



If we want to generate **n convolutional layers**,
we need **n convolutional kernels**.

Pytorch implementation

Conv2d

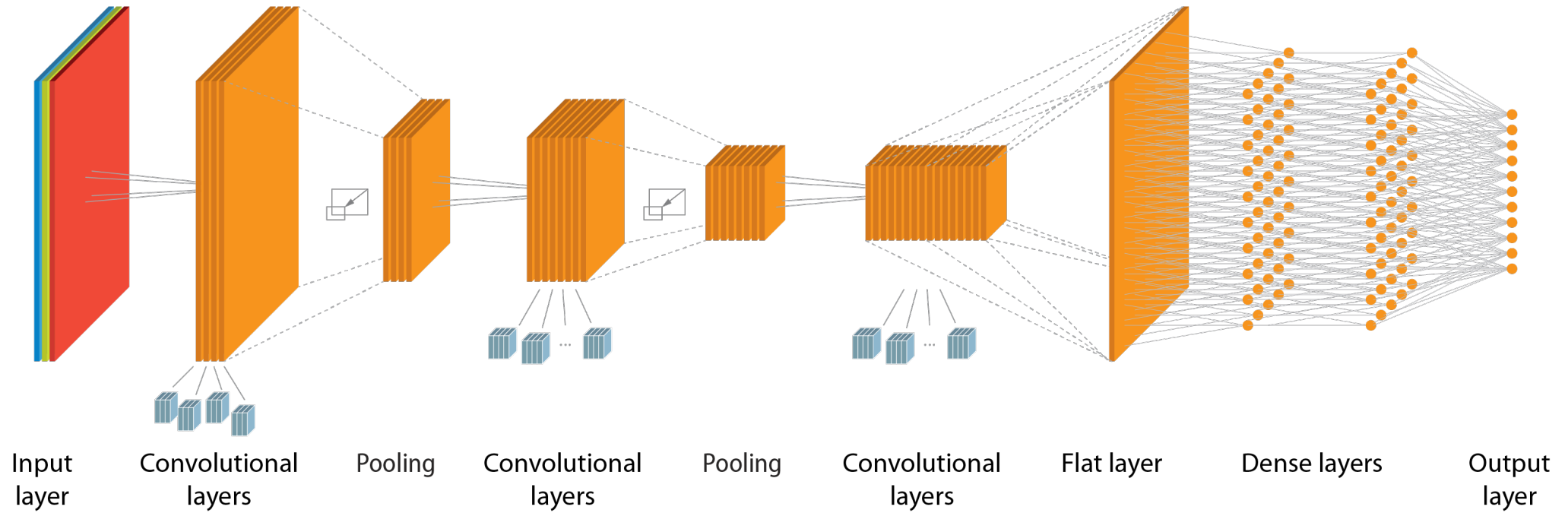
```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1,  
groups=1, bias=True, padding_mode='zeros', device=None, dtype=None) \[SOURCE\]
```

Applies a 2D convolution over an input signal composed of several input planes.

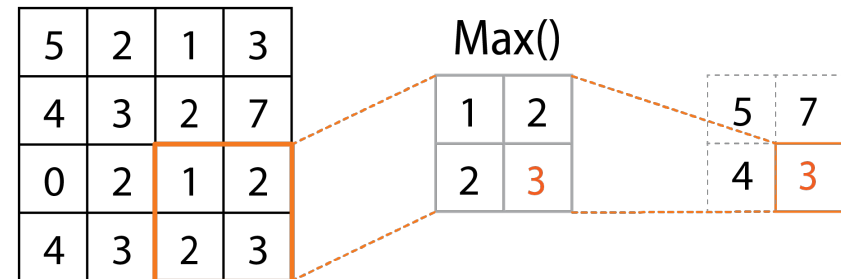
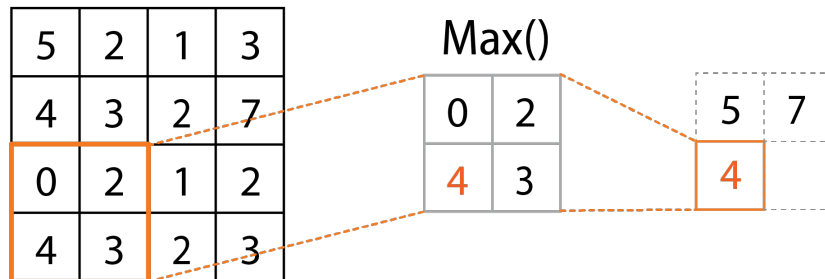
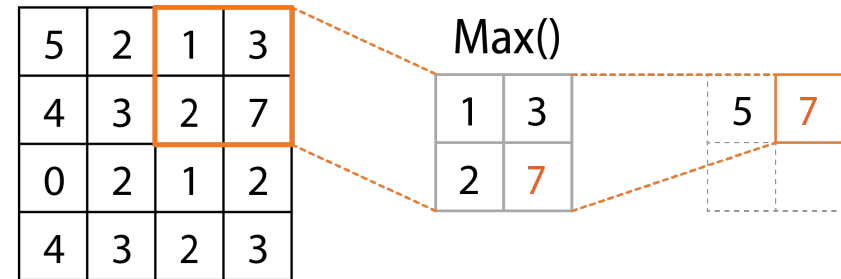
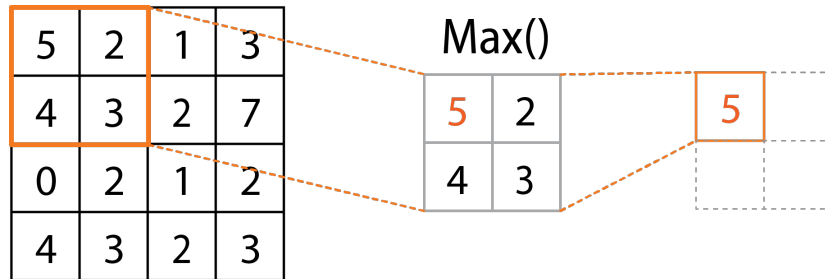
In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

Convolutional Neural Networks (CNN)



Max Pooling



It is possible to set the window size, padding and strides.
By default, the strides correspond to the size of the window.

A window (2,2) generates an image twice as small.

Pooling

MaxPool2d

```
CLASS torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False,  
                        ceil_mode=False) \[SOURCE\]
```

Applies a 2D max pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W) , output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

$$out(N_i, C_j, h, w) = \max_{m=0, \dots, kH-1} \max_{n=0, \dots, kW-1} input(N_i, C_j, stride[0] \times h + m, stride[1] \times w + n)$$

AvgPool2d

```
CLASS torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False,  
                        count_include_pad=True, divisor_override=None) \[SOURCE\]
```

Applies a 2D average pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W) , output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

$$out(N_i, C_j, h, w) = \frac{1}{kH * kW} \sum_{m=0}^{kH-1} \sum_{n=0}^{kW-1} input(N_i, C_j, stride[0] \times h + m, stride[1] \times w + n)$$



Other typical layers

- Activation layers
- Dropout layers randomly sets input units to 0 with a given frequency at each step during training. Helps prevent overfitting.
- Batch normalization: applies a transformation that maintains the mean output close to 0 and the output standard deviation close to 1. Works differently during training and during inference. Fasten the training process.
- Fully connected = MLP

Dropout

```
CLASS torch.nn.Dropout(p=0.5, inplace=False) [SOURCE]
```

BatchNorm2d

```
CLASS torch.nn.BatchNorm2d(num_features, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True, device=None, dtype=None) [SOURCE]
```

Applies Batch Normalization over a 4D input.

4D is a mini-batch of 2D inputs with additional channel dimension. Method described in the paper [Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift](#).

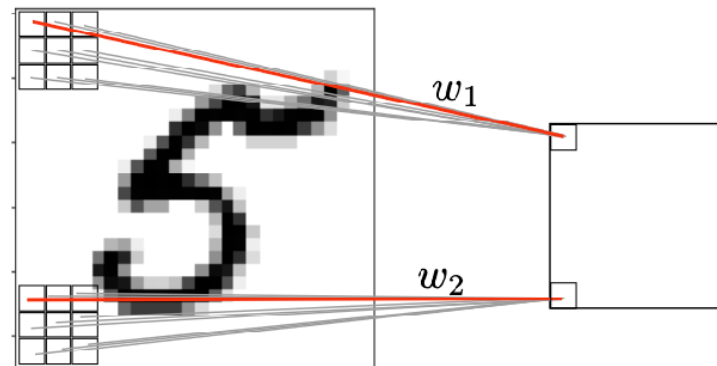
$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Additional ressource

See Slides "Normalization" and "Dropout".

Backpropagation in CNNs

Same concept as for MLP: multivariable chain rule, with weight sharing constraint.

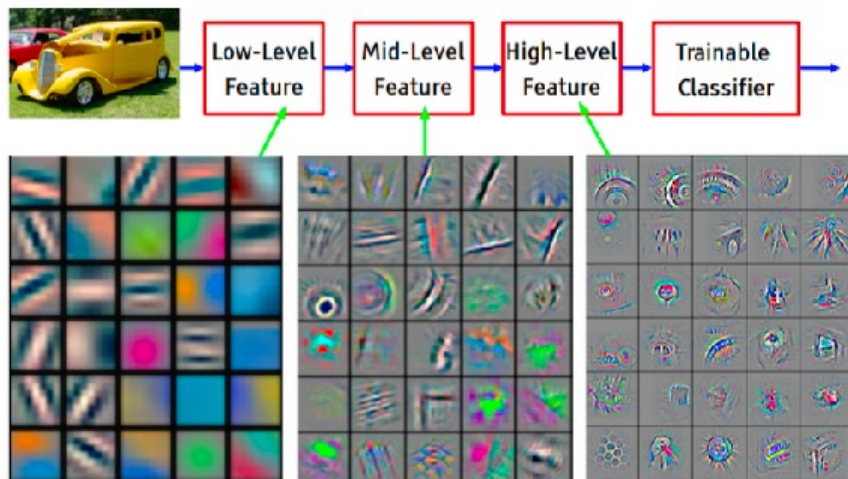
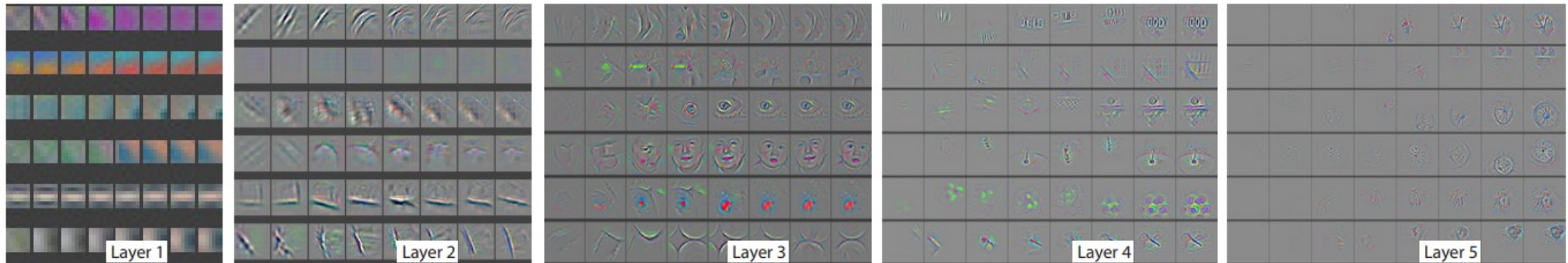


$$w_1 = w_2 = w_1 - \frac{\eta}{2} \left(\frac{\partial \mathcal{L}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial w_2} \right)$$

Additional resource

See slides "Optimization for deep Learning" and "weight initialization"

Visualization of features



"Visualizing and Understanding Convolutional Networks" (Zeiler & Fergus, 2014)

- 
- 1 What _{are} **Convolution kernels** ?
 - 2 What are Convolutional networks ?
 - 3 Applications of CNN

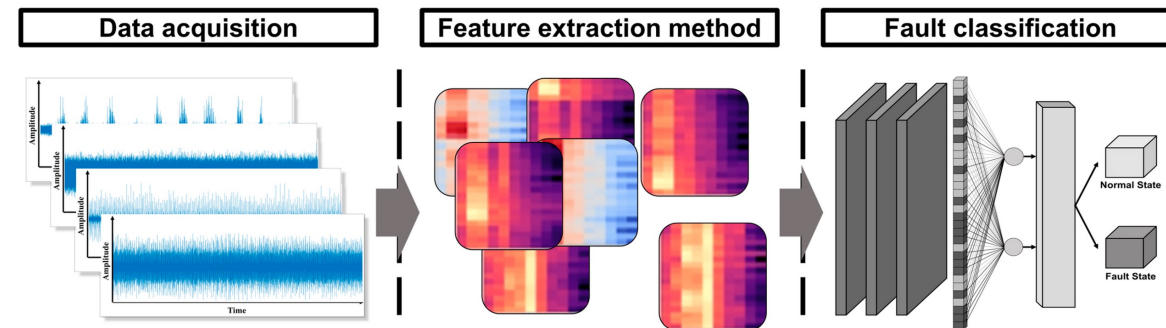
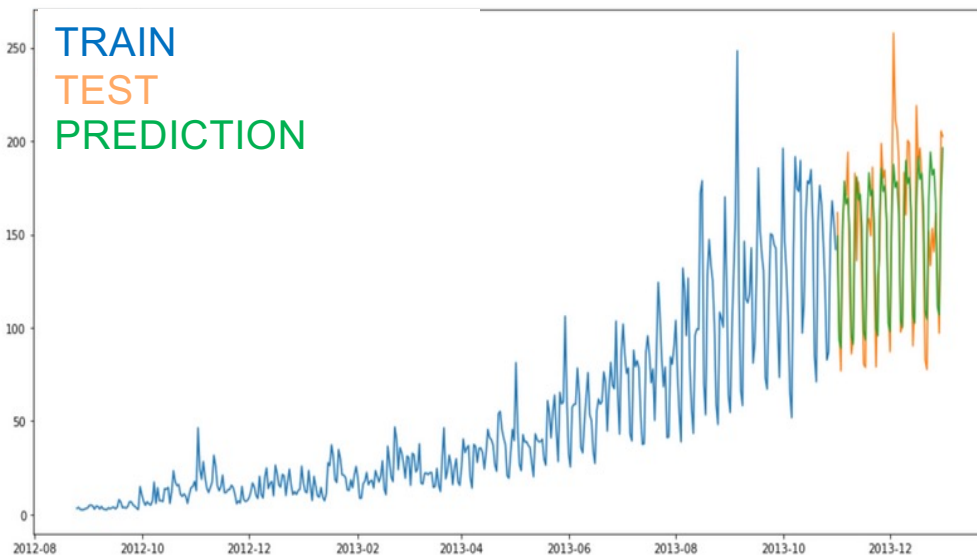
Use of CNN with 1D datasets

Time series predictions : electrocardiogram, weather forecast, traffic flow predictions.

Signal identification

Structural damage identification

Deep Learning for Time Series Forecasting



Sources : Seong and King 2024, Sensors

Li et al. 2021, *IEEE transactions on neural networks and learning systems*

Use of CNN with 2D datasets

- Image classification
- Object detection (one or 2 stages detection)
- Image segmentation (semantic, instance and panoptic segmentations)
- Face recognition
- Landmarking



Classification

cat, 0.93



Detection

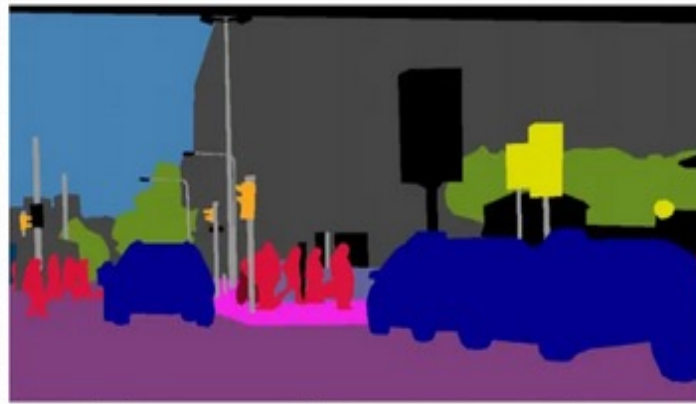


Segmentation

Photo de Loan sur Unsplash



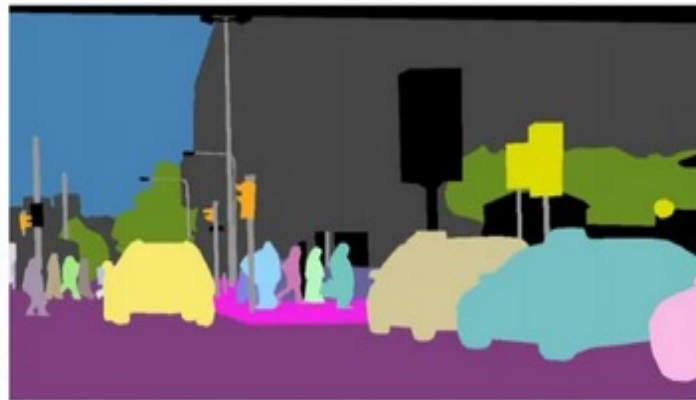
(a) image



(b) semantic segmentation



(c) instance segmentation



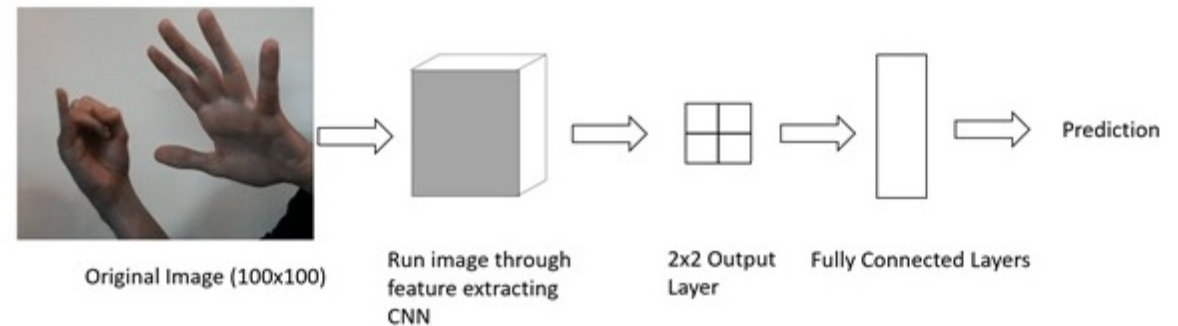
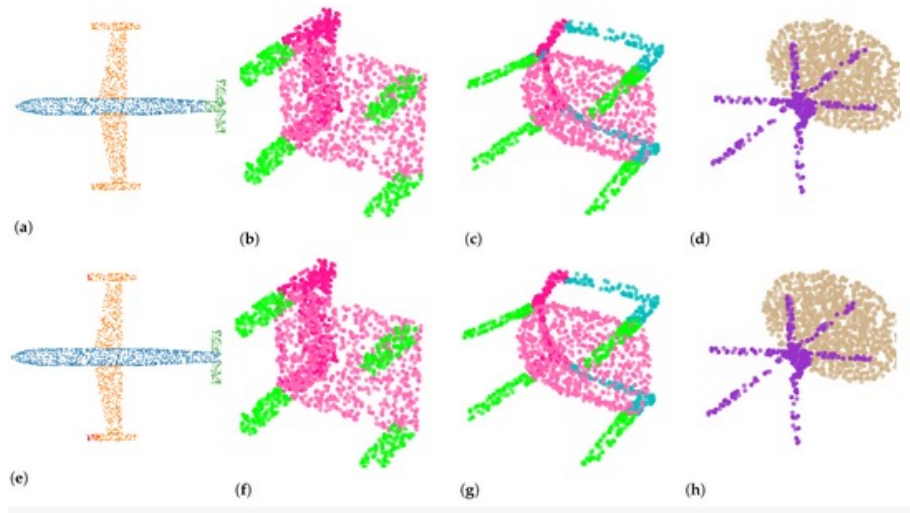
(d) panoptic segmentation

Semantic vs Instance vs Panoptic: Choosing Image Segmentation

<https://www.labellerr.com/blog/semantic-vs-instance-vs-panoptic-which-image-segmentation-technique-to-choose/>

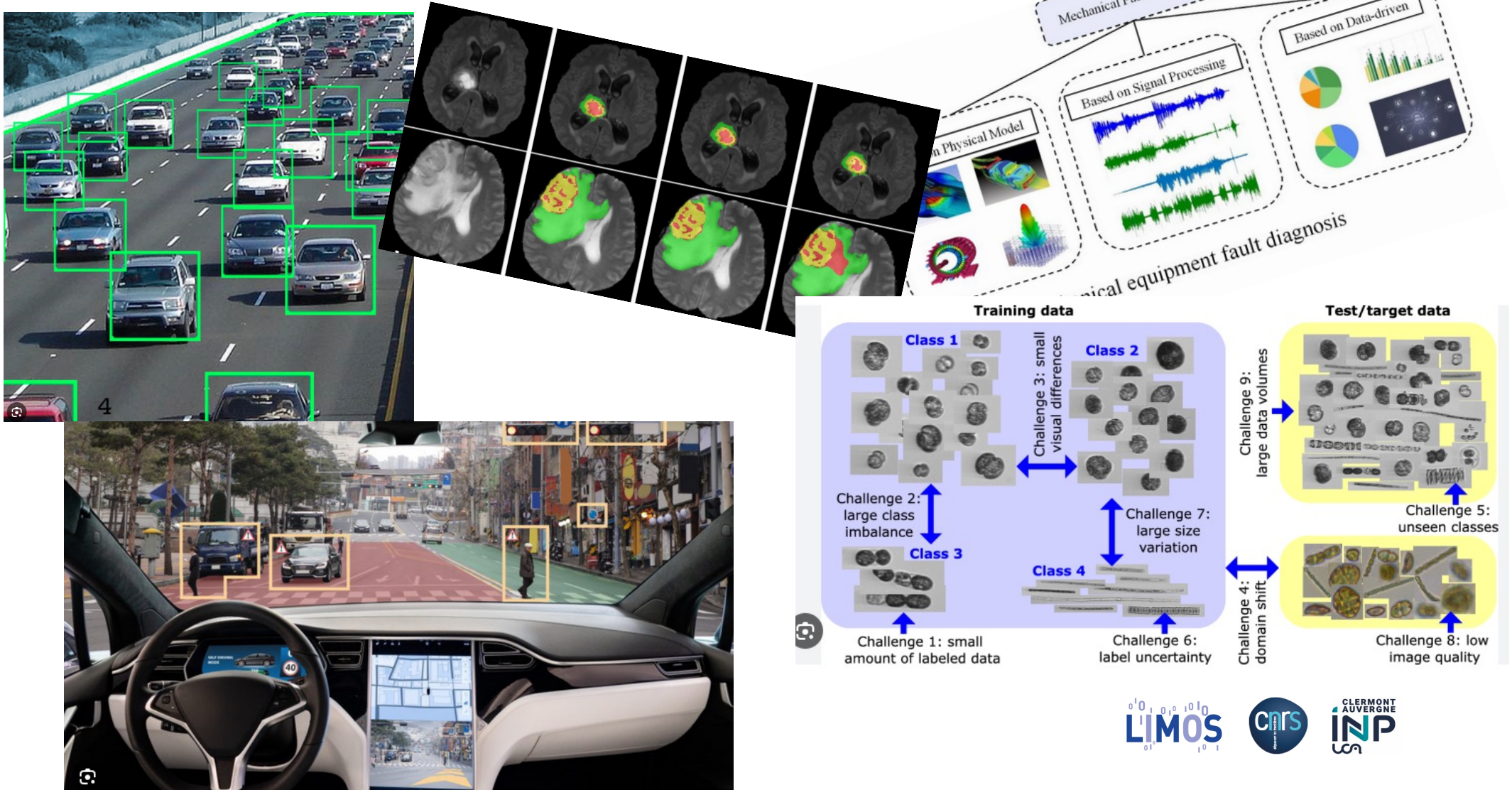
Use of CNN with 3D datasets

- Human action recognition (videos analysis) ex. sign language translation
- Object recognition/detection : in videos, point cloud images, 3D objects



Sources : <https://beetlebox.org/sign-language-recognition-hand-object-detection-using-r-cnn-and-yolo/>
Lai and Law 2023 Remote Sensing

CNN applications : in all fields !



Implementation

```
class CNN(nn.Module):  
    def __init__(self):  
        super(CNN, self).__init__()  
        self.conv1 = nn.Conv2d(1, 32, 5, 1)  
        self.conv2 = nn.Conv2d(32, 64, 5, 1)  
        self.dropout = nn.Dropout(dropout)  
        self.fc1 = nn.Linear(1024, 1024)  
        self.fc2 = nn.Linear(1024, 10)
```

Layer type definition

```
    def forward(self, x):  
        x = self.conv1(x)  
        x = F.relu(x)  
        x = F.max_pool2d(x, 2)  
        x = self.conv2(x)  
        x = F.relu(x)  
        x = F.max_pool2d(x, 2)  
        x = torch.flatten(x, 1)  
        x = self.fc1(x)  
        x = F.relu(x)  
        x = self.dropout(x)  
        x = self.fc2(x)  
        output = F.softmax(x, dim=1)  
        return output
```

CNN architecture